Praktikum 5
Register B dan Adder

NRP	:	1. 3223600038
		2. 3223600041
		3. 3223600048
		4. 3223600050
		5. 3223600058
Nama	:	1. Karolus Wira Wicaksana
		2. Akmal Rafiuddin
		3. Moch. Faris Afif
		4. Labib Al Afaf
		5. Ihsanta Zaki Sanjaya
Kelas	:	2 – D4 Teknik Komputer B
Mata kuliah	:	Praktikum Desain dan Organisasi Komputer
Tanggal	:	Rabu, 16 April 2025
Dosen	:	Dr. Setiawardhana, S.T., M.T.

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 5
Register B dan Adder

5.1. Tujuan

a. Mahasiswa dapat menjelaskan dan membuat program Register
b. Mahasiswa dapat menjelaskan dan membuat program Adder

5.2. Dasar Teori

A. Register

Register B adalah register bufer yang lain diantara register-register bufer yang ada. Register ini digunakan dalam operasi aritmetik. Sinyal LB yang rendah dan tepi positif dari sinyal detak akan mengisikan kata pada bus W ke dalam register B. Keluaran dua keadaan dari register B kemudian menggerakkan Arithmetic Logic Unit, memasukkan bilangan yang akan dijumlahkan dengan atau dikurangkan dari isi akumulator.

B. Adder

SAP-1 menggunakan sebuah Adder atau penjumlah-pengurang disingkat ADD dengan menggunakan komplemen-2. Bila Su berharga rendah, maka keluaran jumlah dari penjumlah-pengurang dalam Gambar 1 adalah:

S = A + B

Apalagi Su tinggi, keluarannya berupa selisih:

A = A + B’

Komplemen-2 ekivalen dengan perubahan tanda dalam bilangan desimal atau menjadi negatif.

Rangkaian penjumlah-pengurang bersifat asinkron (tidak diatur oleh sinyal detak). ini berarti isi keluarannya akan berubah bila terjadinya perubahan pada kata-kata masukan. Bilamana Eu tinggi, isi yang bersangkutan akan muncul pada bus W.

5.3. Tugas Pendahuluan

a. Tuliskan tujuan praktikum.
   1. Mahasiswa dapat menjelaskan dan membuat program Register
   2. Mahasiswa dapat menjelaskan dan membuat program Adder

b. Apa yang anda ketahui tentang Accumulator?

   Akumulator disingkat dengan A adalah sebuah register buffer yang menyimpan jawaban sementara selama komputer beroperasi. Akumulator mempunyai dua macam keluaran. Keluaran dua keadaan secara langsung diteruskan ke bagian Arithmetic Logic Unit. Keluaran tiga-keadaan dikirimkan kepada bus W. Karena itu kata 8-bit dari akumulator secara terus menerus menggerakkan rangkaian Arithmetic Logic Unit;

dan kata yang sama juga muncul pada bus W bilamana EA tinggi.

c. Apa yang anda ketahui tentang Register?

Register adalah sebuah komponen penyimpanan data dalam sistem digital yang dapat menyimpan data sementara untuk digunakan oleh prosesor atau sistem lainnya. Register biasanya memiliki ukuran bit tertentu (misalnya 8-bit, 16-bit, atau 32-bit) dan berfungsi untuk menyimpan data atau hasil sementara dari perhitungan, pengambilan input, atau sebagai tempat penyimpanan status sistem. Register dapat diakses dengan cepat dan digunakan dalam berbagai operasi seperti penyimpanan sementara data, pengontrolan status, atau pengindeksan data dalam prosesor.

d. Apa yang anda ketahui tentang Adder?

SAP-1 menggunakan sebuah Adder atau penjumlah-pengurang disingkat ADD dengan menggunakan komplemen-2. Bila Su berharga rendah, maka keluaran jumlah dari penjumlah-pengurang dalam Gambar 1 adalah:

S = A + B

Apalagi Su tinggi, keluarannya berupa selisih:

A = A + B’

Komplemen-2 ekivalen dengan perubahan tanda dalam bilangan desimal atau menjadi negatif.

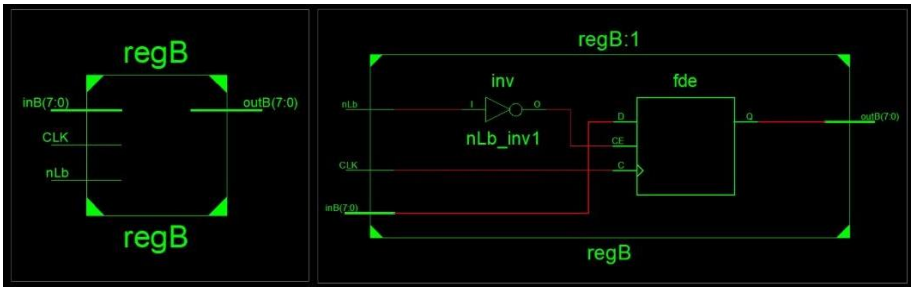
Rangkaian penjumlah-pengurang bersifat asinkron (tidak diatur oleh sinyal detak). ini berarti isi keluarannya akan berubah bila terjadinya perubahan pada kata-kata masukan. Bilamana Eu tinggi, isi yang bersangkutan akan muncul pada bus W.

5.4. Percobaan

a. Membuat program Register B

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity regB is
   Port ( CLK : in  STD_LOGIC;
          nLb : in  STD_LOGIC;
          inB : in  STD_LOGIC_VECTOR (7 downto 0);
          outB : out  STD_LOGIC_VECTOR (7 downto 0));
end regB;

architecture Behavioral of regB is
begin
 process (CLK)
 begin
  if rising_edge(CLK) then
   if nLb = '0' then outB <= inB;
   end if;
  end if;
 end process;
end Behavioral;
```

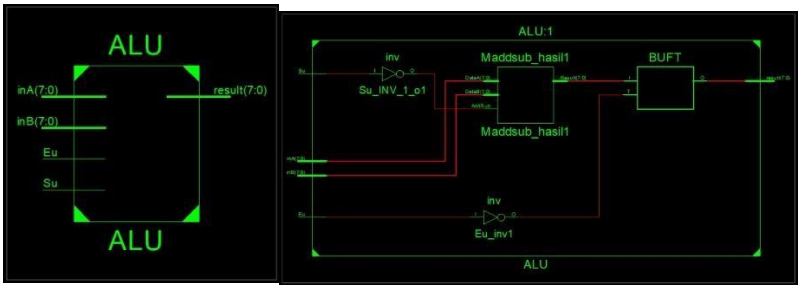
RTL Schematic



b. Membuat program Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity ALU is
   Port ( Su : in  STD_LOGIC;
          Eu : in  STD_LOGIC;
          inA : in  STD_LOGIC_VECTOR (7 downto 0);
          inB : in  STD_LOGIC_VECTOR (7 downto 0);
          result : out  STD_LOGIC_VECTOR (7 downto 0));
end ALU;
architecture Behavioral of ALU is
 signal hasil: STD_LOGIC_VECTOR (7 downto 0);
begin
 hasil <= inA+inB when Su='0' else inA-inB;
result <= hasil when Eu='1' else "ZZZZZZZZ";
end Behavioral;
```

RTL Schematic



5.5. Latihan

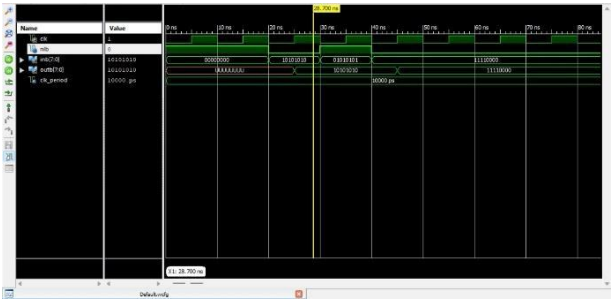
a. Buatlah simulasi program Register B

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity tb_regB is
-- Testbench tidak memiliki port
end tb_regB;

architecture behavior of tb_regB is

   -- Komponen yang akan diuji
   component regB
      Port (
         CLK : in  STD_LOGIC;
         nLb : in  STD_LOGIC;
         inB : in  STD_LOGIC_VECTOR (7 downto 0);
         outB : out  STD_LOGIC_VECTOR (7 downto 0)
      );
   end component;
```

```
-- Sinyal internal untuk menghubungkan ke unit yang diuji
   signal CLK   : STD_LOGIC := '0';
   signal nLb    : STD_LOGIC := '1';
   signal inB   : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
   signal outB   : STD_LOGIC_VECTOR(7 downto 0);

   -- Clock period
   constant CLK_period : time := 10 ns;

begin

   -- Instansiasi unit yang diuji (UUT)
   uut: regB
      port map (
         CLK => CLK,
         nLb => nLb,
         inB => inB,
         outB => outB
      );

   -- Proses clock
   CLK_process :process
   begin
      while true loop
         CLK <= '0';
         wait for CLK_period/2;
         CLK <= '1';
         wait for CLK_period/2;
      end loop;
   end process;
```

```
  -- Stimulus untuk menguji fungsionalitas regB
   stim_proc: process
   begin
      -- Inisialisasi awal
      wait for 20 ns;

      -- Kasus 1: Load data 10101010 ke regB
      inB <= "10101010";
      nLb <= '0';  -- Enable load
      wait for CLK_period;

      -- Kasus 2: Ganti data input tapi nLb di-disable
      inB <= "01010101";
      nLb <= '1';  -- Tidak load
      wait for CLK_period;

      -- Kasus 3: Load data baru
      inB <= "11110000";
      nLb <= '0';  -- Enable load
      wait for CLK_period;

      -- Selesai
      wait for 20 ns;
      wait;
   end process;

end behavior;
```

Timing Diagram



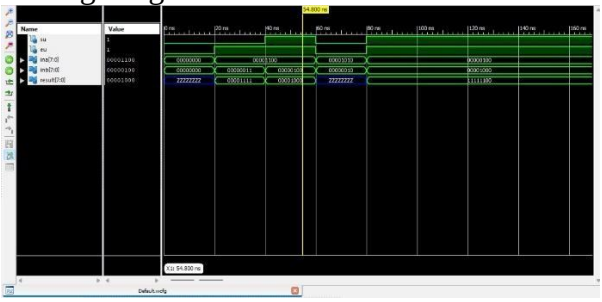
b. Buatlah simulasi program Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity tb_ALU is
-- Testbench tidak memiliki port
end tb_ALU;

architecture behavior of tb_ALU is

   -- Komponen yang diuji
   component ALU
      Port (
         Su   :  in  STD_LOGIC;
         Eu     : in  STD_LOGIC;
         inA  :  in  STD_LOGIC_VECTOR  (7  downto  0);
         inB  :  in  STD_LOGIC_VECTOR  (7  downto  0);
         result : out STD_LOGIC_VECTOR (7 downto 0)
      );
   end component;
```

```
-- Sinyal untuk menghubungkan ke ALU
   signal   Su   : STD_LOGIC := '0';
   signal   Eu   : STD_LOGIC := '0';
   signal inA    : STD_LOGIC_VECTOR(7 downto 0) := (others =>
'0');
   signal inB    : STD_LOGIC_VECTOR(7 downto 0) := (others =>
'0');
   signal result : STD_LOGIC_VECTOR(7 downto 0);

begin

   -- Instansiasi unit yang diuji
   uut: ALU
      port map (
         Su     => Su,
         Eu     => Eu,
         inA    =>   inA,
         inB    =>   inB,
         result => result
      );
-- Proses stimulus untuk pengujian
   stim_proc: process
   begin
      -- Tunggu sebentar untuk inisialisasi
      wait for 20 ns;

      -- Tes 1: Penjumlahan (A + B) dengan Eu aktif
      inA <= "00001100";     -- 12
      inB <= "00000011";     -- 3
      Su  <= '0';            -- Penjumlahan
      Eu  <= '1';            -- Enable output
      wait for 20 ns;
-- Tes 2: Pengurangan (A - B) dengan Eu aktif
      inA <= "00001100";     -- 12
      inB <= "00000100";     -- 4
      Su  <= '1';            -- Pengurangan
      Eu  <= '1';            -- Enable output
      wait for 20 ns;
```

```
-- Tes 3: Output di-disable (Eu = 0)
        inA <= "00001010";     -- 10
        inB <= "00000010";     -- 2
        Su  <= '0';            -- Penjumlahan
        Eu  <= '0';            -- Disable output
        wait for 20 ns;

        -- Tes 4: Hasil negatif (komplemen 2)
        inA <= "00000100";     -- 4
        inB <= "00001000";     -- 8
        Su  <= '1';            -- 4 - 8 = -4 (komplemen 2)
        Eu  <= '1';
        wait for 20 ns;

        wait; -- Selesai
    end process;

end behavior;
```

Timing Diagram



5.6. Analisa

a. Program Register B

Kode VHDL di atas adalah implementasi dari register B (binary) adder. Register B adder adalah jenis adder yang dapat menambahkan dua bilangan biner. Kode VHDL ini mendefinisikan entity bernama Adder yang memiliki empat input dan satu output. Input pertama, Su, adalah bit kontrol pengatur operasi adder. Ketika Su bernilai '0', maka adder akan melakukan operasi penjumlahan. Sedangkan jika Su bernilai '1', maka adder akan

melakukan operasi pengurangan. Input kedua, Eu, adalah bit kontrol pengatur output adder. Ketika Eu bernilai '1', maka output adder akan menjadi hasil dari operasi penjumlahan atau pengurangan. Sedangkan jika Eu bernilai '0', maka output adder akan menjadi 'ZZZZZZZZ'. Input ketiga dan keempat adalah inA dan inB, masing-masing merupakan dua bilangan biner yang akan dioperasikan oleh adder. Kedua bilangan ini memiliki panjang 8 bit, dari bit 0 hingga bit 7. Output dari entity Adder adalah result, yang merupakan hasil dari operasi penjumlahan atau pengurangan dari inA dan inB. Output ini juga memiliki panjang 8 bit.

Pada blok architecture, terdapat sebuah signal bernama hasil yang memiliki panjang 8 bit. Signal ini akan menyimpan hasil operasi penjumlahan atau pengurangan dari inA dan inB, tergantung pada nilai Su. Kemudian, nilai dari signal hasil akan diteruskan ke output result jika nilai Eu adalah '1'. Jika Eu adalah '0', maka output result akan bernilai 'ZZZZZZZZ'. Kode VHDL ini menggunakan library IEEE dan dua package, yaitu STD_LOGIC_1164 dan STD_LOGIC_UNSIGNED. Library dan package tersebut digunakan untuk mendefinisikan tipe data dan fungsi yang digunakan dalam kode VHDL.

b. Program Adder

Kode VHDL di atas adalah implementasi dari Adder, yaitu penjumlah biner. Adder adalah sebuah komponen digital yang menerima dua bilangan biner sebagai input dan mengeluarkan hasil penjumlahan dari kedua bilangan tersebut. Kode VHDL ini mendefinisikan entity Adder yang memiliki empat input dan satu output. Input pertama, Su, adalah bit kontrol pengatur operasi adder. Ketika Su bernilai '0', maka adder akan melakukan operasi penjumlahan. Sedangkan jika Su bernilai '1', maka adder akan melakukan operasi pengurangan. Input kedua, Eu, adalah bit kontrol pengatur output adder. Ketika Eu bernilai '1', maka output adder akan menjadi hasil dari operasi penjumlahan atau pengurangan. Sedangkan jika Eu bernilai '0', maka output adder akan menjadi 'ZZZZZZZZ'. Input ketiga dan keempat adalah inA dan inB, masing-masing merupakan dua bilangan biner yang akan dioperasikan oleh adder. Kedua bilangan ini memiliki panjang 8 bit, dari bit 0 hingga bit 7. Output dari entity Adder adalah result, yang merupakan hasil dari operasi penjumlahan atau pengurangan dari inA dan inB. Output ini juga memiliki panjang 8 bit.

Pada blok architecture, terdapat sebuah signal bernama hasil yang memiliki panjang 8 bit. Signal ini akan menyimpan hasil operasi penjumlahan atau pengurangan dari inA dan inB, tergantung pada nilai Su. Kemudian, nilai dari signal hasil akan diteruskan ke output result jika nilai Eu adalah '1'. Jika Eu adalah '0', maka output result akan bernilai 'ZZZZZZZZ'. Kode VHDL ini menggunakan library IEEE dan dua package, yaitu STD_LOGIC_1164 dan STD_LOGIC_UNSIGNED. Library dan package tersebut digunakan untuk mendefinisikan tipe data dan fungsi yang digunakan dalam kode VHDL.

5.7. Kesimpulan

Percobaan di atas mencakup dua kode VHDL; register dan adder. Kode register digunakan untuk menyimpan nilai dan kode adder digunakan untuk melakukan operasi penjumlahan atau pengurangan biner dari dua nilai 8-bit berdasarkan sinyal kontrol. Kode register mendefinisikan sebuah entity dengan sinyal clock input, sinyal reset, dan sinyal output nilai. Ketika sinyal reset diaktifkan, nilai output akan diatur ke nilai default. Ketika sinyal clock diaktifkan, nilai input akan disimpan ke dalam nilai output. Kode adder mendefinisikan sebuah entity dengan empat input (dua nilai biner 8-bit dan dua sinyal kontrol) dan satu output (nilai biner 8-bit). Dua sinyal kontrol menentukan apakah adder melakukan operasi penjumlahan atau pengurangan dan apakah nilai output valid atau tidak. Hasil operasi disimpan dalam sebuah sinyal dan kemudian diteruskan ke output jika sinyal output valid. Secara keseluruhan, kedua kode VHDL yang diberikan digunakan untuk mengimplementasikan register dan adder. Kode register digunakan untuk menyimpan nilai dan kode adder digunakan untuk melakukan operasi penjumlahan atau pengurangan biner dari dua nilai 8-bit berdasarkan sinyal kontrol. Kedua kode menggunakan library dan paket standar untuk mendefinisikan tipe data dan fungsi yang digunakan dalam kode.